

Metanet 详细设计

本文档为 Metanet 设计文档体系的第三层：算法、数据结构、协议细节。

文档体系：- [整体设计](#) — 两产品生态、三层架构、界面划分 - [概念设计](#) — 产品定位、核心理念、设计原则 - [系统设计](#) — 节点架构、合约、支付通道 - 详细设计（本文档） — 共识、挖矿、结算协议细节 - [测试设计](#) — 测试用例设计

本文档补充 Metanet Chain 的精确协议细节、Script 设计和经济参数，与系统设计各节对应。

一、存储合约 Bitcoin Script

存储合约是 Owner 与 Metanet Node 之间的 HTLC 变体，锁定 Token 作为存储费用：

StorageDeal UTX0（第 k 期）：

```
OP_IF
  // Metanet Node 路径: <node_sig> <proof_data> 1
  <node_pubkey> OP_CHECKSIGVERIFY           // 验证 Metanet Node 签名
  OP_SHA256 <expected_hash_k> OP_EQUALVERIFY // 验证存储证明
  // expected_hash_k = SHA256(MerkleProof_k || chunk[challenge_k % N])
  OP_TRUE                                     // Metanet Node 领取当期费用
OP_ELSE
  // Owner 退款路径: <owner_sig> 0
  <expire_block> OP_CHECKLOCKTIMEVERIFY OP_DROP
  <owner_pubkey> OP_CHECKSIG                 // Owner 取回 Token
OP_ENDIF
```

确定性挑战机制：

// 合约创建时：Owner 预计算所有 N 期的证明

contract_txid = <合约交易的 TxID，创建后确定>

```
for k := 0; k < N; k++ {
  // 确定性挑战：使用 contract_txid 作为熵源
  challenge_k = SHA256(contract_txid || uint32_le(k))
  chunk_index = uint32(challenge_k[0:4]) % num_chunks

  // 计算该期的 Merkle 证明
```

```

proof_k = MerkleProof(data_tree, chunk_index)
expected_hash_k = SHA256(proof_k || chunks[chunk_index])

// 写入第 k 个 UTXO 的 Script
utxo_k.script = ... OP_SHA256 <expected_hash_k> OP_EQUAL ...
}

```

合约参数：

value:	锁定的 MNT Token (当期费用)
expire_block:	证明提交截止区块高度
expected_hash_k:	第 k 期预计算的证明哈希 (确定性, 嵌入 UTXO)
owner_pubkey:	Owner 公钥
node_pubkey:	Metanet Node 公钥

合约生命周期：

1. Owner 创建 StorageDeal 交易, 锁定 N 期费用 (N 个 UTXO)
2. 每期挑战由 contract_txid 确定性派生 (无需链上随机源)
3. Metanet Node 提交证明, 解锁当期 UTXO
4. 若 Metanet Node 未在截止前提交证明, Owner 可取回该期费用
5. 全部期满: 合约自然结束, Owner 可续签

设计原理：“随机性”来源于 contract_txid 在合约创建前不可预测 (依赖交易内容的哈希)。合约一旦上链, 所有期数的挑战均可确定性计算, 与 UTXO 不可变性完全兼容。Metanet Node 无法提前知道 contract_txid, 因此无法仅存储被挑战的 chunk 而丢弃其他数据。

二、存储证明 Bitcoin Script

存储证明交易验证 Metanet Node 持有正确数据：

存储证明 Script (StorageProof)：

```

// 输入: Metanet Node 提交 Merkle proof
<node_sig> <chunk_data> <merkle_siblings> <chunk_index>

// 验证逻辑 (Script):
1. OP_SHA256 <chunk_data>                                → chunk_hash
2. 从 chunk_index 和 merkle_siblings 逐层计算:
   for each sibling in merkle_siblings:
     if bit(chunk_index, i) == 0:
       OP_CAT <sibling> OP_SHA256                        → 左拼接
     else:
       OP_SWAP OP_CAT OP_SHA256                          → 右拼接
3. 最终结果与合约中存储的 merkle_root 对比

```

简化实现（当前阶段）：

- Script 只验证 `OP_SHA256(proof_data) == expected_hash`
- 完整 Merkle 验证在链下进行，结果哈希上链
- 任何人可链下重放验证

完整 Script 验证（远期）：

- 利用 BSV 大 Script 能力，在 Script 内完成完整 Merkle 验证
- 无需信任链下验证者

三、ECDH 双层加密流程

// 伪代码: Owner 为 Metanet Node 重加密数据

```
func ReencryptForProvider(  
    ownerPrivKey *ec.PrivateKey, // D_node  
    providerPubKey *ec.PublicKey, // P_provider  
    encryptedData []byte,        // 已用 Method 42 加密的数据  
) (providerEncrypted []byte, merkleRoot []byte, err error) {  
  
    // Step 1: ECDH 派生 Metanet Node 专用密钥  
    sharedSecret := ECDH(ownerPrivKey, providerPubKey)  
    providerKey := KDF(sharedSecret) // SHA256(shared_x || "storage")  
  
    // Step 2: 用 Metanet Node 专用密钥再加密 (双层)  
    // 注意: 不解密原始数据, 直接在密文上再加一层  
    nonce := random(12) // AES-256-GCM nonce  
    providerEncrypted = AES256GCM_Encrypt(providerKey, nonce, encryptedData)  
  
    // Step 3: 分块构建 Merkle 树  
    chunks := SplitIntoChunks(providerEncrypted, CHUNK_SIZE) // CHUNK_SIZE = 256KB  
    leaves := make([][]byte, len(chunks))  
    for i, chunk := range chunks {  
        leaves[i] = SHA256(chunk)  
    }  
    merkleRoot = BuildMerkleTree(leaves)  
  
    return providerEncrypted, merkleRoot, nil  
}  
  
// 验证存储证明  
func VerifyStorageProof(  
    merkleRoot []byte,  
    chunkIndex int,  
    chunkData []byte,  
    siblings [][]byte,  
) bool {  
    hash := SHA256(chunkData)
```

```

for i, sibling := range siblings {
    if chunkIndex >> i & 1 == 0 {
        hash = SHA256(append(hash, sibling...))
    } else {
        hash = SHA256(append(sibling, hash...))
    }
}
return bytes.Equal(hash, merkleRoot)
}

```

关键性质:

- 每个 Metanet Node 的密文不同 (不同 ECDH 共享密钥)
- Metanet Node 无法解密内容 (不知道 Owner 的第一层密钥)
- Metanet Node 无法伪造证明 (需要实际持有数据才能回答 Merkle 挑战)
- Owner 可验证任何 Metanet Node 的证明 (持有所有 ECDH 密钥)

性能说明: 大文件 (如 1GB) 双层加密开销显著 — N 个 Metanet Node 需 N 次独立 AES-GCM 加密 + N 次网络上传。优化方向: (1) 延迟加密 — Metanet Node 请求时 Owner 实时提供 Provider 密钥, Metanet Node 自行加密; (2) 分片级加密 — 仅对被挑战的 chunk 执行双层加密。

四、支付通道协议

支付通道交易结构:

1. 开通道 (Funding TX):

Input: User 的 UTXO (BSV 或 Token)

Output: 2-of-2 多签 (User + Metanet Node)

Value: channel_capacity

Script: `OP_2 <user_pubkey> <node_pubkey> OP_2 OP_CHECKMULTISIG`

2. 链下更新 (Commitment TX, 不广播):

Input: Funding TX output

Output 0: Node → node_balance

Output 1: User → user_balance

其中 $\text{node_balance} + \text{user_balance} = \text{channel_capacity}$

每次 x402 请求:

node_balance += price

user_balance -= price

双方签名新的 Commitment TX

3. 关通道 (Settlement TX):

任一方广播最新 Commitment TX

争议解决详细机制：

- 状态版本：每次通道更新附带递增 sequence_number
- 撤销密钥：每次状态更新，双方交换上一状态的 revocation_key
- 旧状态广播：若一方广播旧版本 (sequence_number < latest):
 - 另一方在 T 个块 (CSV 时间锁) 内使用 revocation_key 提交惩罚交易
 - 惩罚：广播旧状态方损失全部通道余额
- 正常关闭：双方签署最终状态，无需等待时间锁
- 使用 OP_CHECKSEQUENCEVERIFY 实现时间锁

通道参数：

min_deposit: 10000 sat (最小存款)
max_duration: 144 blocks (约 1 天, 最长通道寿命)
dispute_window: 6 blocks (争议窗口)
update_frequency: 每次 x402 请求

五、x402 支付通道 HTTP 协议扩展

新增 HTTP Headers：

请求端：

X-Accept-Channel: true # 客户端支持通道支付
X-Channel-ID: <funding_txid>:<vout> # 已有通道 ID
X-Payment-Voucher: <base64(signed_ctx)> # 签名的通道更新

响应端：

X-Accept-Channel: true # 服务端支持通道支付
X-Channel-Price: <sats_per_kb> # 通道内价格 (可低于单次价)
X-Channel-Min-Deposit: <sats> # 最低通道存款
X-Channel-Balance: <remaining_sats> # 通道剩余余额
X-Channel-Expiry: <block_height> # 通道的过期高度

通道内支付流程：

1. Client: GET /data/{hash} (X-Channel-ID + X-Payment-Voucher)
2. Server: 验证签名, 更新本地余额
3. Server: 200 + data + X-Channel-Balance
4. 无需等待链上确认, 即时完成

通道开启流程：

1. Client: POST /channel/open (funding_tx)
2. Server: 验证 funding_tx, 返回 channel_id
3. Client: 广播 funding_tx 到链上
4. Server: 等待确认后激活通道

通道关闭流程：

1. Client: POST /channel/close 或 Server 主动关闭
2. 广播最新 Commitment TX
3. 等待 dispute_window 后结算

错误处理:

- 余额不足: 402 + X-Channel-Balance: 0 + X-Channel-TopUp-Required: true
- 通道过期: 402 + X-Channel-Expired: true
- 签名无效: 400 Bad Request

六、BSV 锚定交易格式

BSV 锚定交易 (Anchor TX):

Output 0: OP_RETURN <metanet_anchor_flag> <data>

Output 1: change (返回 Owner/矿工)

data 格式:

version: uint8 = 0x01
anchor_flag: bytes4 = "MNTA" (Metanet Anchor)
start_height: uint32 = Metanet Chain 起始块高度
end_height: uint32 = Metanet Chain 结束块高度
merkle_root: bytes32 = 这批 Metanet Chain 块的 Merkle root
block_count: uint16 = 块数量 (通常 100)
prev_anchor_txid: bytes32 = 上一次锚定的 BSV txid (链式引用)

锚定频率:

每 100 Metanet Chain 块 → 1 笔 BSV 交易

≈ 每 ~16.7 小时一次 (100 × 10 分钟)

BSV 矿工费: ~1 sat/byte × ~120 bytes ≈ 120 sat

验证:

任何人可验证: 给定 Metanet Chain 块范围, 计算 Merkle root, 对比 BSV 上的锚定

防长程攻击: 攻击者需同时篡改 BSV 上的锚定记录

七、合并挖矿技术细节

合并挖矿 (AuxPoW) 流程:

1. 矿工构建 BTC/BSV 区块时:

- 在 coinbase 交易中嵌入: OP_RETURN <aux_magic> <sub_chain_block_hash>
- aux_magic = "MNMP" (Metanet Merged PoW)

2. 矿工同时构建 Metanet Chain 区块:

- 正常的 Metanet Chain 区块 header
- 额外字段: parent_chain_header + coinbase_tx + merkle_branch

3. Metanet Chain 验证:

- 验证 parent_chain_header 满足 Metanet Chain 难度
- 验证 coinbase_tx 在 parent_chain_header 的 Merkle 树中
- 验证 coinbase_tx 包含正确的 sub_chain_block_hash
- 验证 sub_chain_block_hash == SHA256d(sub_chain_block_header)

AuxPoW Block Header:

```
// 标准 Metanet Chain header
version:          uint32
prev_hash:        bytes32
merkle_root:      bytes32
timestamp:        uint32
bits:             uint32
nonce:            uint32

// AuxPoW 附加数据
parent_header:    bytes80   (BTC/BSV block header)
parent_coinbase_tx: bytes    (包含 aux_magic 的 coinbase)
coinbase_branch:  []bytes32 (coinbase 在父链的 Merkle 分支)
```

难度调整:

- 独立于 BTC/BSV 的难度
- 每 2016 块调整 (与 Bitcoin 相同周期)
- 目标: 平均 10 分钟出块

八、Token 经济参数

MNT Token 参数:

总供应量: 21,000,000 MNT
最小单位: 1 satoshi = 0.00000001 MNT
区块奖励: 50 MNT (初始)
减半周期: 210,000 块
区块时间: ~10 分钟 (目标)
难度调整: 每 2016 块

减半时间表:

Era 0:	块 0 - 209,999	50.0 MNT/块	总计 10,500,000
Era 1:	块 210,000 - 419,999	25.0 MNT/块	总计 5,250,000
Era 2:	块 420,000 - 629,999	12.5 MNT/块	总计 2,625,000
Era 3:	块 630,000 - 839,999	6.25 MNT/块	总计 1,312,500
...			

(与 Bitcoin 减半曲线完全一致)

区块大小:

初始上限: 32 MB (与 BSV 一致)

未来: 可通过矿工投票提升

交易费:

最低费率: 1 sat/byte

与 BSV 费率策略一致

创世块:

时间戳: (主网启动时确定, 当前为实验性参数)

message: "Metanet: Decentralized CDN on Bitcoin"

奖励接收者: 基金会多签地址 (3-of-5)

九、带宽加权 PoW (远期优化)

目标: 激励矿工同时提供存储和带宽, 不只是算力。

带宽加权难度调整:

$$\text{adjusted_difficulty} = \text{base_difficulty} \times \text{bandwidth_factor}$$
$$\text{bandwidth_factor} = 1 / (1 + \alpha \times \text{proven_bandwidth} / \text{target_bandwidth})$$

α = 带宽权重系数 (初始 0, 可通过矿工投票调整)

proven_bandwidth:

- Metanet Node 在过去 N 块内的 x402 交易总量 (链上可验证)
- 更多 x402 = 更多带宽贡献 = 更低挖矿难度

效果:

- $\alpha = 0$: 纯 PoW (初始状态, 与 Bitcoin 相同)
- $\alpha > 0$: 提供更多 CDN 带宽的矿工获得挖矿优势
- 渐进调整: 网络成熟后逐步提高 α

注意:

- 这是远期优化, 初始版本使用纯 SHA256 PoW
- 需要充分的 x402 交易量才有意义
- 防作弊: x402 交易必须有对应的数据哈希和签名, 无法伪造